

Service API & Logging Contract

This document defines the minimum API contract, request flow, response format, and logging structure expected from all teams.

Teams are free to implement the services in Python or Node.js, but the endpoints, payloads, and log formats must follow this specification.

Service A

Port:

3001

Public Access:

Accessible through Nginx

Service Name:

service-a

GET /health

Purpose:

Health check endpoint.

Example Request:

```
curl http://localhost/service-a/health
```

Expected Response:

```
{
  "service": "service-a",
  "status": "healthy",
  "port": 3001,
```

```
{
  "message": "Hello service-a listening on 3001"
}
```

HTTP Status:

200 OK

GET /greet-service-b

Purpose:

Starts the complete request flow.

Flow:

```
Service A
↓
Service B
↓
Service C
↓
Service A callback
```

Expected Response:

```
{
  "request_id": "<request-id>",
  "status": "success",
  "message": "Request completed successfully"
}
```

HTTP Status:

200 OK

POST /greeting-rcvd

Purpose:

Receives callback from Service C.

Expected Request Body:

```
{
  "request_id": "<request-id>",
  "source_service": "service-c",
  "message": "Greeting processed",
  "timestamp": "2025-01-01T10:00:00Z"
}
```

Expected Response:

```
{
  "status": "received"
}
```

HTTP Status:

200 OK

Service B

Port:

3002

Public Access:

Internal Only

Service Name:

service-b

GET /health

Expected Response:

```
{
  "service": "service-b",
}
```

```
{
  "status": "healthy",
  "port": 3002,
  "message": "Hello service-b listening on 3002"
}
```

HTTP Status:

200 OK

GET /greet

Purpose:

Receives request from Service A and forwards to Service C.

Expected Headers:

X-Request-ID

Expected Response:

```
{
  "request_id": "<request-id>",
  "status": "forwarded",
  "target": "service-c"
}
```

HTTP Status:

200 OK

Service C

Port:

3003

Public Access:

Internal Only

Service Name:

service-c

GET /health

Expected Response:

```
{
  "service": "service-c",
  "status": "healthy",
  "port": 3003,
  "message": "Hello service-c listening on 3003"
}
```

HTTP Status:

200 OK

GET /greet-c

Purpose:

Receives request from Service B.

Expected Headers:

X-Request-ID

Expected Actions:

1. Log request.
2. Call Service A callback endpoint.
3. Return success response.

Expected Response:

```
{
  "request_id": "<request-id>",
  "status": "processed",
  "callback_sent": true
}
```

HTTP Status:

200 OK

Required Request Flow

The following sequence must occur:

```
Client
↓
Nginx
↓
Service A
↓
Service B
↓
Service C
↓
Service A Callback
```

The same request_id must be visible throughout the entire flow.

Request ID Requirements

Every request must contain:

X-Request-ID

If missing:

Service A should generate one.

Example:

a3b9d61f-22f2-48f1-91f8-bcb6d7b91b21

The same value must be propagated to:

Service A
Service B
Service C
Service A callback

Structured Log Format

All services must produce structured JSON logs.

Each log entry must contain:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "service": "service-a",
  "event": "request_received",
  "request_id": "abc-123",
  "method": "GET",
  "path": "/greet-service-b",
  "status": 200
}
```

Required fields:

Field	Required
timestamp	Yes
service	Yes
event	Yes
request_id	Yes
path	Yes
status	Yes

Success Event Logs

Example:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "service": "service-b",
  "event": "request_forwarded",
  "request_id": "abc-123",
  "target": "service-c",
  "status": 200
}
```

Callback Logs

Example:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "service": "service-a",
  "event": "callback_received",
  "request_id": "abc-123",
  "source_service": "service-c",
  "status": 200
}
```

Error Logs

All errors must be logged.

Example:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "service": "service-c",
  "event": "request_failed",
  "request_id": "abc-123",
  "path": "/greet-c",
  "status": 500,
  "error": "downstream_timeout"
}
```

404 Logs

Any unknown route must:

1. Return HTTP 404
2. Create a structured log entry

Example:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "service": "service-b",
  "event": "route_not_found",
  "request_id": "abc-123",
  "path": "/invalid-route",
  "status": 404
}
```

Nginx Log Requirements

Nginx access logs should be structured and include:

```
{
  "timestamp": "2025-01-01T10:00:00Z",
  "request_id": "abc-123",
  "method": "GET",
  "path": "/service-a/greet-service-b",
  "status": 200,
  "upstream": "service-a:3001",
  "request_time": 0.021
}
```

Validation Commands

The following commands should succeed:

```
curl http://localhost/service-a/health
curl http://service-b.internal:3002/health
curl http://service-c.internal:3003/health
```

Trigger full flow:

```
curl http://localhost/service-a/greet-service-b
```

The reviewer should be able to locate the same request_id in:

- Nginx logs
- Service A logs
- Service B logs
- Service C logs
- Service A callback logs

and reconstruct the full request journey.